CampusRide

Real-Time Campus Mobility & Ride Management Platform

Design Document · Cult Open Projects 2026 · Problem Statement 2

IIT Roorkee · Ashish Kumar Agrawal

1 · Problem Understanding

IIT Roorkee's campus spans a large area and depends heavily on e-rickshaws for last-mile transport, yet ride requests and driver availability are coordinated through informal, fragmented channels. CampusRide replaces this with a centralised, real-time digital system connecting passengers and drivers.

The platform is built around the engineering challenges the brief identifies as central:

- **Real-time communication** — live ride status, driver availability, and assignment notifications over WebSockets.
- **Ride-assignment workflow** — a request must be assignable to exactly one driver, with no double-booking.
- **State synchronisation** — a single, consistent ride lifecycle visible to passenger, driver and operations.
- **Geospatial handling** — campus zones, distance-based fares, ETAs, and a live map.
- **Multi-user coordination & analytics** — dashboards for drivers and administrators.

Scope was deliberately kept to the core ride-management problem rather than replicating every commercial ride-hailing feature, while still delivering all six bonus capabilities (live map, scheduling, digital payments, demand analytics, ML demand forecasting, advanced dashboard).

2 · System Architecture

CampusRide is a two-tier full-stack application with a real-time channel layered over a conventional REST API.

2.1 High-level components

Layer	Technology	Responsibility
Client (SPA)	React 18, Vite, Tailwind, Recharts, React-Leaflet	Passenger / Driver / Tracking / Admin UIs; Socket.IO client
Transport	REST (HTTP) + Socket.IO (WebSocket)	CRUD & queries over REST; live events over WebSocket
Server	Node.js 22, Express, Socket.IO	Auth, ride lifecycle, dispatch engine, analytics, forecasting
Data	SQLite (built-in node:sqlite)	Users, drivers, rides, ratings, payments, audit logs

2.2 Real-time dispatch flow

1. A passenger emits `ride:request`; the server creates the ride and broadcasts `ride:new` to the `drivers` room. 2. Online drivers see the request instantly and may emit `ride:accept`. 3. Acceptance runs a conditional UPDATE that succeeds for only the first driver; others receive `"ride already taken"`. 4. The passenger (in the per-ride room) receives `ride:update` for every transition and the driver streams `driver:location`, relayed as `ride:location`.

2.3 Room model

- `user:<id>` — a private room every socket joins on connect.
- `drivers` — all currently-online drivers; receives broadcast dispatch.
- `ride:<id>` — the passenger plus the assigned driver; receives status & location.

Every socket authenticates with its JWT during the handshake before any event is processed, so identity and role are trusted server-side.

3 · Database Schema

Six normalised tables with foreign keys and CHECK constraints enforce integrity at the storage layer. Status and ride-type values are constrained so an invalid state can never be written.

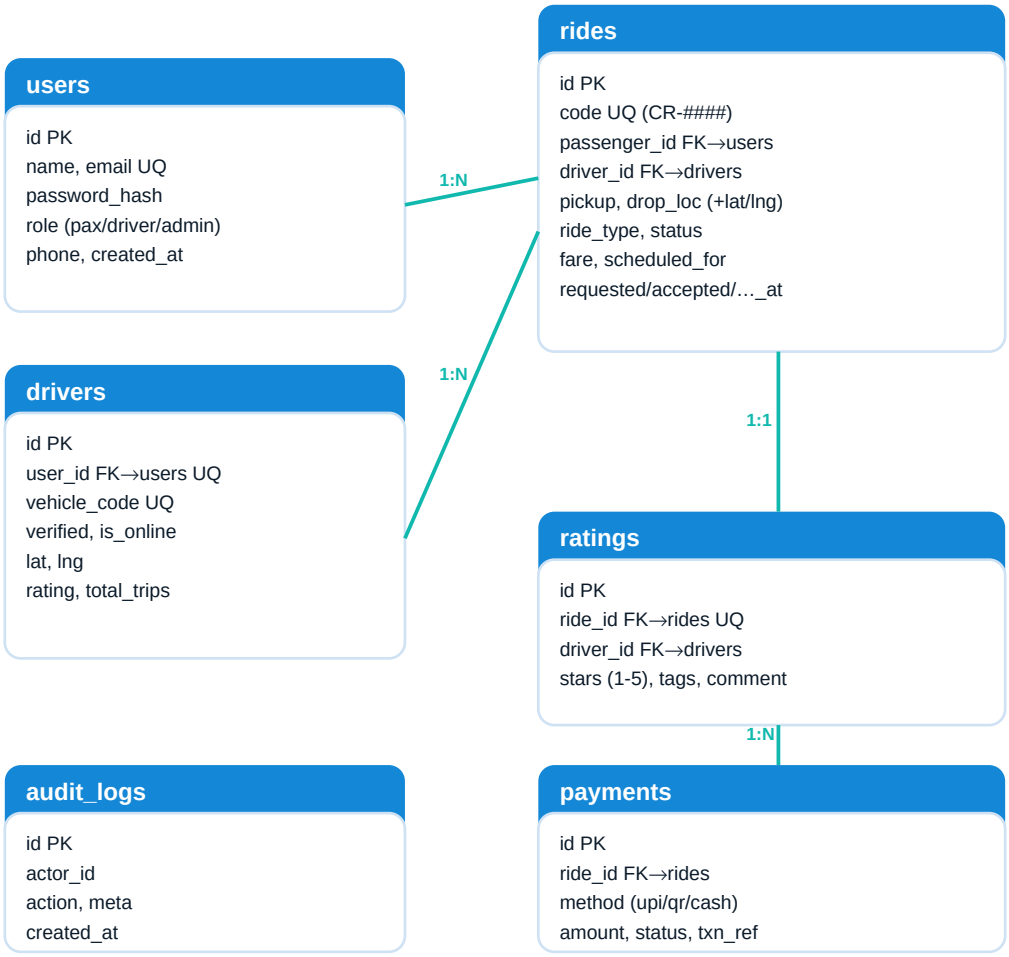
Table	Key columns	Notes
users	id, email (UQ), role	role ∈ {passenger, driver, admin}
drivers	user_id (FK,UQ), vehicle_code (UQ), is_online, rating	one profile per driver user
rides	code (UQ), passenger_id (FK), driver_id (FK), status, fare	status & timestamps per stage
ratings	ride_id (FK,UQ), driver_id (FK), stars	one rating per ride; recomputes avg
payments	ride_id (FK), method, amount, txn_ref	method ∈ {upi, qr, cash}
audit_logs	actor_id, action, meta	key actions for traceability

3.1 Ride lifecycle (state machine)

`requested` → `accepted` → `arrived` → `in_progress` → `completed`, with `cancelled` reachable from any active state. Each transition writes a timestamp (`accepted_at, started_at, completed_at, cancelled_at`), giving an auditable, consistent history and enabling the analytics queries.

Indexes on rides.status, rides.passenger_id, rides.driver_id and drivers.is_online keep the live dispatch and dashboard queries fast as data grows.

4 · Entity Relationship Diagram



Cardinalities: a **user** (passenger) has many **rides**; a **driver** fulfils many **rides**; each **ride** has at most one **rating** (1:1) and may have one or more **payments**. All relationships are enforced with foreign keys.

5 · API Overview

5.1 REST endpoints

Method	Path	Purpose
POST	/api/auth/register	Create passenger / driver account (JWT)
POST	/api/auth/login	Authenticate, return JWT
GET	/api/auth/me	Current user + driver profile
GET	/api/rides/quote	Fare + ETA preview (solo / share / cargo)
GET	/api/rides/open	Open requests for drivers
GET	/api/rides/active	Live rides for the ops board
GET	/api/rides/mine	Role-aware ride history
GET	/api/rides/scheduled	Upcoming scheduled rides
PATCH	/api/drivers/availability	Set online / offline
GET	/api/drivers/me/dashboard	Driver KPIs, earnings, ratings split
POST	/api/ratings	Submit rating; recompute driver average
POST	/api/payments	Simulated UPI / QR payment (+ upi:// link)
GET	/api/analytics/overview	KPI cards
GET	/api/analytics/peak-hours	Rides per hour (24h)
GET	/api/analytics/heatmap	Pickups per zone
GET	/api/analytics/forecast	14-day demand forecast (ML)

5.2 Socket.IO events

- `driver:online / driver:offline` — availability toggle (broadcasts driver count).
- `ride:request` → `ride:new` — passenger requests; broadcast to online drivers.
- `ride:accept` → `ride:taken` — atomic claim; remove from other drivers' lists.
- `ride:status` → `ride:update` — lifecycle transitions pushed to the ride room.
- `driver:location` → `ride:location` — live GPS relayed to the passenger.

6 · Design Decisions

6.1 Atomic single-driver assignment

Rather than locking or a queue, acceptance is a single conditional statement: `UPDATE rides SET driver_id=?, status='accepted' WHERE id=? AND status='requested' AND driver_id IS NULL`. SQLite guarantees only one writer mutates the row, so exactly one driver wins; the rest get a clean rejection. This is simple, race-free and verified by an automated test where a second driver is denied with "ride already taken."

6.2 Built-in SQLite for reproducibility

The data layer uses Node 22's built-in `node:sqlite` module instead of a native add-on or external database server. A grader needs only Node ≥ 22.5 — no compiler, no `node-gyp`, no Postgres/Mongo — making the project install and run identically everywhere. A thin `tx()` helper wraps multi-statement transactions.

6.3 Rooms over manual fan-out

Socket.IO rooms target updates precisely (one ride's participants, or all online drivers) without the server tracking socket lists by hand, keeping the dispatch code small and correct.

6.4 Transparent forecasting model

Demand forecasting uses an interpretable decomposition — OLS linear trend × multiplicative weekly seasonal index, with an EWMA correction on recent residuals and an ~80% band from the fit RMSE. It is fast, dependency-free and explainable, and exposes the same JSON contract so it can later be swapped for an ARIMA / Prophet / gradient-boosted service without touching the API or UI.

6.5 Fair, flat pricing

Fares are deterministic (base + distance, with a per-ride-type multiplier and a ■12 floor) — no surge — matching the campus context and giving passengers a predictable quote before they book.